Design Patterns - Lab 7
Observer

Goal
Learn how to implement push messages on server-side using the Observer pattern.

Tasks

1. Create an index.html under the following location /resources/public/index.html with the following content; it is a HTML5 + JS that leverages on server-sent events to receive events pushed from the server-side [1, 2].

```
<html lang="en">
<body>

<h1>All Books</h1>
<a href="javascript:void(0)" onclick="sendRequest()">Fetch Books</a>
<div id="sseDiv"></div>

<script>
	function sendRequest() {
	if(typeof(EventSource) !== "undefined") {
	const source = new EventSource(`http://localhost:8080/books-sse`);
	source.onmessage = function(event) {
		const data = JSON.parse(event.data);
		document.getElementById("sseDiv").innerHTML += data.id +
"<br>";
	};
	} else {
	document.getElementById("sseDiv").innerHTML = "Your browser does not
support server-sent events.";
	}
   }
</script>

</body>
</html>
```

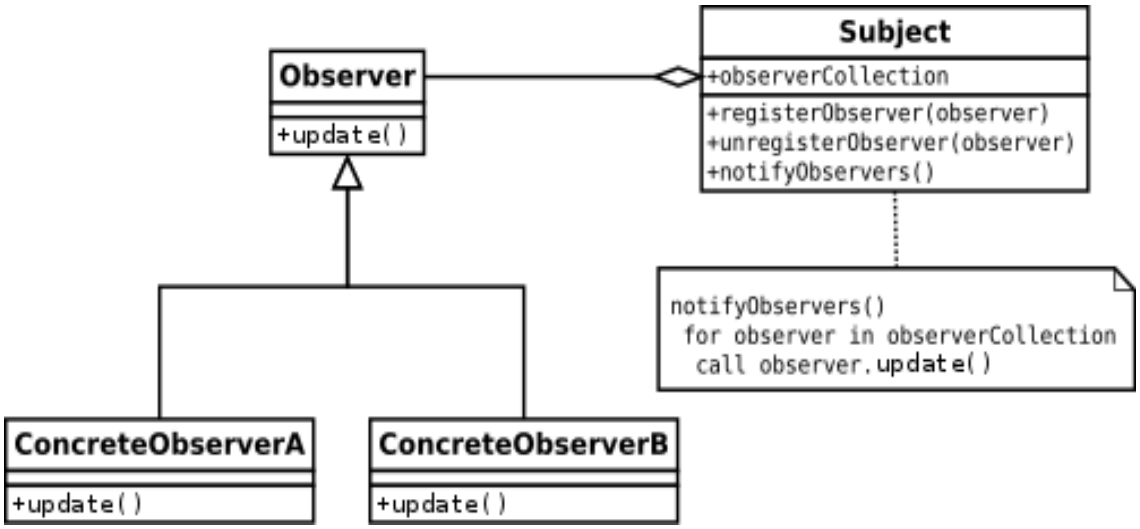
2. Create a new controller (BooksSseController) and add the GET method

```
@RequestMapping("/books-sse")
public ResponseBodyEmitter getBooksSse() {
    final SseEmitter emitter = new SseEmitter(0L);
    allBooksSubject.attach(new SseObserver(emitter));
    return emitter;
}
```

3. In the BooksController.newBook method, which is handling POST /books, change the state of the `allBooksSubject`, that should notify all observers about the newly created book, which in turn will push an event to the index.html page

```
@PostMapping
public String newBook(@RequestBody NewBookRequest newBookRequest) {
    Book book = createBook(newBookRequest);
    book = booksRepository.save(book);
    allBooksSubject.add(book);
    return "Book saved [" + book.getId() + "] " + book.getTitle();
}
```

4. allBooksSubject is the ConcreteSubject participant of the Observer pattern, while SseObserver class and Observer interface are the Concrete Observer and Observer participants, respectively. Following the Observer pattern diagram, add the necessary interfaces and their concrete implementations.



5. The SseObserver.update() method implementation will send a properly JSON-formatted message through SseEmitter to the “listening” client

```
@Override
public void update(Book book) {
    emitter.send(book, MediaType.APPLICATION_JSON);
}
```

6. More details about using SSE events with Spring framework can be found in [3]

7. If CORS-related errors, then you need to allow CORS requests from all/selected sources. In order to enable CORS requests from any origin to any endpoint in the application, add the following code to your Spring main class

```
@Bean
public WebMvcConfigurer corsConfigurer() {
   return new WebMvcConfigurer() {
       @Override
       public void addCorsMappings(CorsRegistry registry) {
           registry.addMapping("/**");
       }
   };
}
```

   More details about enabling CORS in [4]

8. References

[1] https://html.spec.whatwg.org/multipage/server-sent-events.html
[2] https://www.w3schools.com/html/html5_serversentevents.asp
[3] https://www.baeldung.com/spring-server-sent-events
[4] https://www.baeldung.com/spring-cors